

TECHNICAL ASSESSMENT SUBMISSION

DETECT TECHNOLOGIES | TAP-JOB-ID-2507

Candidate Name:	Suprit Lenkenavar
Institute:	TAP Academy
Deadline:	Thursday, 28-05-2026, 11:00 AM
Status:	All 3 Problems Fully Completed & Verified

Important Note regarding Video Quality & File Sizes:

To guarantee high accuracy in our computer vision pipelines (such as face detection and license plate character recognition/OCR), we intentionally avoided heavy compression on our sample videos. Compressing these video files directly below the Form's 10MB limit would introduce severe pixelation and compression noise, directly degrading the model detection and reading rates. Therefore, we hosted the full-quality original videos on Google Drive to allow the reviewer to evaluate the actual, uncompromised capability of our systems.

Submission Links

GitHub Repository: Contains all version-controlled source code, requirements, and documentation.	GitHub Repo Link https://github.com/suprit-4618/...Assignment.git
Google Drive Folder: Contains the original high-resolution input and output demo videos (bypassing the 10MB upload limit).	Google Drive Link https://drive.google.com/drive/folders/1vQ6xd...

Interactive Review (Google Colab)

To make evaluation as simple as possible, interactive Google Colab notebooks have been pre-configured. The reviewer can run the pipelines on GPU with a single click:

Problem 2: Face Blur	Open Face Blur Notebook
Problem 3: License Plates	Open Plate Detection Notebook

Summary of Completed Assignments

1. Move Zeros to the Right (Easy)

- Implementation: $O(n)$ Time / $O(n)$ Space algorithm keeping non-zero relative order.
- Location: `problem1_move_zeros/solution.py`
- Verification: Passed all 10 edge-case test suites (negative numbers, empty arrays, etc.).

2. Face Blur on Video (Medium)

- Implementation: OpenCV Haar Cascade detectors (Frontal + Profile) with IoU deduplication and dynamic Gaussian blur padding.
- Location: `problem2_face_blur/face_blur.py`
- Output: 12,642 verified real-time blurred face detections across 1,080 frames of street crowd footage.

3. License Plate Detection & OCR (Hard)

- Implementation: Haar cascade + contour aspect-ratio filtering, four-point perspective correction, and EasyOCR engine.
- Location: `problem3_number_plate_detection/number_plate_detector.py`
- Optimization: CPU skip-frame processing engine (runs OCR every N frames; copies logs on intermediate frames).
- Output: Extremely high-accuracy reads (e.g., N28-800928, confidence 1.00) logged in `sample_output.json`.